Digital representasjon

Gruppe 3 - Justas baltrukonis

Fra sist

- Hva betyr ting
  - 0100 0001

I dag

- Notasjon
- Bits – hva er det?
  - Verdier
- Omregning
- LMC!!!

Notasjon

- Annet (programmering)
  - Binær
    - 0b4512
  - Desimal
    - 255
  - Heksadesimal
    - 0xff

Notasjon

- Base
  - Binær
    - 2
  - Desimal
    - 10
  - Heksadesimal
    - 16

Notasjon

$$255_{10} \rightarrow 255$$
$$11111111_{2} \rightarrow \texttt{0b11111111}$$
$$FF_{16} \rightarrow \texttt{0x}FF$$

Heksadesimal

- 0 1 2 3 4 5 6 7 8 9 A B C D E F (hex)
- 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 (desimal)
- 16 ?
  - 10

DECIMAL	HEX	BINARY
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Posisjoner + omregning til desimal

0	1	0	0	0	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	$2^6 \cdot 1 = 64$	0	0	0	0	$2^1 \cdot 0 = 0$	$2^0 \cdot 1 = 1$

- Grunntall og posisjon - harmonien
- Her kan man bytte grunntallet (2) for hva som helst (som 16)

Omregning fra desimal

- Desimal til binær
- Regner nedover
- Leser oppover
- Her gjelder modulo (%)
  - Hvordan fungerer det?
  - Finne rest

Verdi	Rest
25	1
12	0
6	0
3	1
1	1

Svaret leser vi nedenfra: $25_{10} = 11001_2$

Hva med de andre sin omregning

- Her gjelder modulo også
- Velger et tall i desimal (base 10)
- Finner ut hva man vil til (velger grunntall)
- Deler først på grunntallet – får neste tall
- Så tar man modulo istedenfor å dele (får rest)
  - Samme som å ta opprinnelig tall – (gruntall x heltallet man fikk da man delte)
- Kommer eksempel

Eksempel

241 til hex

$241/16 = 15,0625$ (her beholder vi 15) så $241 - (16 \cdot 15) = 1$

$15/16 = 0$ og får 15 i rest

1 = 1 og 15 = F

Leser nedenfra og opp, da får man $F1_{16}$

Samme med modulo

$241/16 = 15$ for å finne rest $241 mod 16 = 1$ (1)

$15/16 = 0$ og rest blir $15 mod 16 = 15$ (f)

Det mod og % er samme

Eksempel

- 2 måter
  - Dele så gange opp
  - Dele så modulo
- Dette fungerer for alt

241 til hex

$241/16 = 15,0625$ (her beholder vi 15) så $241 - (16 \cdot 15) = 1$

$15/16 = 0$ og får 15 i rest

1 = 1 og 15 = F

Leser nedenfra og opp, da får man $F1_{16}$

Samme med modulo

$241/16 = 15$ for å finne rest $241 mod 16 = 1$ (1)

$15/16 = 0$ og rest blir $15 mod 16 = 15$ (f)

Det mod og % er samme

Toerkomplement

- En måte for datamaskinen å representere negative tall
- Dette finnes og dere burde være klare over det
- 0 = positivt tall
- 1 = negativt tall
- Ganske enkelt
- Skriv tallet med 8 bits
- Inverter
- + 1

Eksempel

toerkomplement

$5 = 00000101$ invertert = 11111010 legg til en (+1) = $11111010(+1)$ = 11111011

$-5 = 11111011$

Dette er på grunn av overflow (på en måte)

- Maskin bits – 000 = 0 (Y2K)
  - Hva skjer med 0 -1?
  - 000 – 1 = 111
  - 001 = 110 så +1 = 111
- Hvordan finne ut?
- Signed vs Unsigned
  - Veldig likt
  - Om det er toerkomplement så er det -128 ellers +128
  - 2^7 kolonnen blir negativ (den første) når den er signed

Invertering av 128, hva skjer da.

$128_{10} = 10000000_2$ (ref tabellen, $2^7 = 128$ dette er den eneste biten som trengs)

$10000000 \rightarrow \text{invert} = 01111111$

$0111111 + 1 = 10000000 = -128$ i toerkomplement

Toerkomplem-entet sine limits

- Overflow kan skje
- Reglene for 8 bits:
  - Unsigned (hvor man ikke tenker på fortegn)
    - 0-255 (vanlig binær)
  - Signed (der fortegnet har noe å si)
    - -128 til 127
    - Mister på en måte en bit
- Prøver man på dette får man int overflow

Konkret eksempel

- Hvilke Hvilke verdier kan *11000001* ha? Med og uten toerkomplement

1	1	0	0	0	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
$2^7 \cdot 1 = 128$	$2^6 \cdot 1 = 64$	0	0	0	0	$2^1 \cdot 0 = 0$	$2^0 \cdot 1 = 1$

$128 + 64 + 1 = 193$

1	1	0	0	0	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-128	64	32	16	8	4	2	1
$-2^7 \cdot 1 = -128$	$2^6 \cdot 1 = 64$	0	0	0	0	$2^1 \cdot 0 = 0$	$2^0 \cdot 1 = 1$

$-128 + 64 + 1 = -63$

- Kan også invertere, plusse på 1, regne ut og sette på minus foran

Oppsummering omgjøring og utregning

toerkomplement

$5 = 00000101$ invertert = 11111010 legg til en (+1) = $11111010(+1) = 11111011$

$-5 = 11111011$

Dette er på grunn av overflow (på en måte)

1	1	0	0	0	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
$2^7 \cdot 1 = 128$	$2^6 \cdot 1 = 64$	0	0	0	0	$2^1 \cdot 0 = 0$	$2^0 \cdot 1 = 1$

$128 + 64 + 1 = 193$

1	1	0	0	0	0	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-128	64	32	16	8	4	2	1
$-2^7 \cdot 1 = -128$	$2^6 \cdot 1 = 64$	0	0	0	0	$2^1 \cdot 0 = 0$	$2^0 \cdot 1 = 1$

$-128 + 64 + 1 = -63$

Oppsummering

- Det dere har nå er veien fra desimal til hva som helst
- Og hva som helst til desimal
- Toerkomplement

- Finnes andre måter, jeg gjorde bare sånt
  - En generell formel kan alltid anvendes, mindre å huske på!

LMC – la oss ta det sammen

Alle instruksjonene i LMC med assemblernotasjon

Kode	Navn	Beskrivelse
0	HLT	Stopper eksekveringen
1	ADD	Adderer verdien i angitt minnelokasjon med akkumulatoren
2	SUB	Subtraherer verdien i angitt minnelokasjon med akkumulatoren
3	STA	Lagrer akkumulatoren i angitt minnelokasjon
4	-	Ikke i bruk
5	LDA	Henter verdi fra minnet til akkumulatoren
6	BRA	Hopper til angitt adresse
7	BRZ	Hopper hvis akkumulatoren er 0
8	BRP	Hopper hvis akkumulatoren er ≥ 0
901	INP	Leser verdi fra input, og legger svaret i akkumulatoren
902	OUT	Skriver ut verdien i akkumulatoren
922	OTC	Skriver ut ASCII-tegn (ikke i boka)

ASCII tabell

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	|
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

www.alpharithms.com